

Smart Coding Exam System

Database documentation

1. Overview

The application uses a MySQL database named `smart_coding_exam`. It stores users, exams, coding questions, test cases, results, code submissions, and activity/audit events.

The backend connection is configured in `backend/src/main/resources/application.properties`. The default database host is `localhost:3306`. Connection values can be supplied through `MYSQL_URL`, `MYSQL_USER`, and `MYSQL_PASSWORD` environment variables.

2. How the database is created

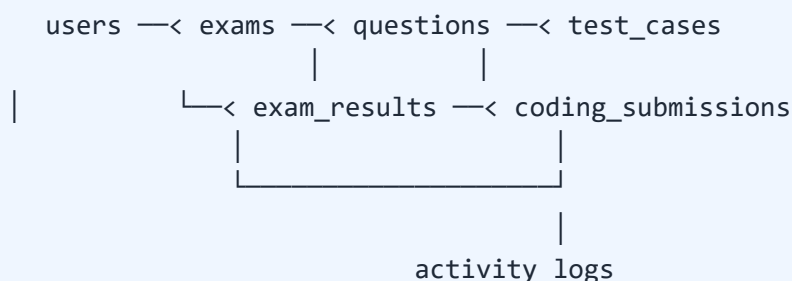
1. Create the MySQL database.
2. Start the Spring Boot backend.
3. Hibernate compares the Java entities with the schema and creates missing tables or compatible columns.

```
CREATE DATABASE IF NOT EXISTS smart_coding_exam
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

The backend setting `spring.jpa.hibernate.ddl-auto=update` keeps the schema aligned with the entity classes without deleting existing data.

Important: `database/schema_mysql.sql` is a useful initial schema reference, but it is older than the current entity model. The active schema is maintained by Hibernate when the backend runs.

3. Database relationships



Each “one-to-many” relationship uses a foreign key. For example, a question belongs to one exam through `questions.exam_id`, while an exam may have many questions.

4. Active application tables

Table	Purpose	Main relationships
users	Admin and student accounts, profile data, role, and BCrypt password hash.	Creates exams; owns results, submissions, and activity logs.
exams	Exam title, description, duration, marks, publish state, creator, and availability times.	Created by a user; contains questions and results.
questions	Coding problem statement, marks, difficulty, language, starter code, and execution limits.	Belongs to an exam; contains test cases.
test_cases	Expected input/output checks, sequence number, weight, and sample-test indicator.	Belongs to a question.
exam_results	One student's overall exam attempt: scores, percentage, pass state, and timestamps.	Links a student and exam; contains code submissions.
coding_submissions	Submitted source code, language, compiler/runtime output, test-case counts, auto score, manual review, and status.	Links an exam result, student, exam, and question.
activity_logs	Audit/proctoring events, including login, tab switch, copy/paste attempt, start, and submission.	Links a student, exam, and optionally an exam result.

5. Data flow

1. An admin creates an exam; a row is stored in `exams`.
2. The admin adds questions and test cases; they are stored in `questions` and `test_cases`.
3. A student starts an exam; the system creates or updates an `exam_results` record.
4. For each coding answer, the system stores a row in `coding_submissions`.
5. Code evaluation updates the submission with output, errors, passed tests, timing, and score.
6. Final marks and percentage are saved in `exam_results`.
7. Important student actions are recorded in `activity_logs`.

6. Integrity and security

- Primary keys use auto-increment numeric IDs.
- Foreign keys enforce links between parent and child records.
- Several child records cascade when their parent exam/question/result is deleted.
- Unique constraints prevent duplicate usernames and duplicate result attempts for the same student, exam, and attempt number.
- Passwords are stored as BCrypt hashes, never plain text.

- Indexes support common lookups such as published exams, exam questions, student submissions, and student results.

7. Older reference-only tables

The SQL reference script also defines attendance and evaluation_results. The present Java entity model does not use these tables. Current code evaluation details are stored directly in coding_submissions.

8. Project file structure

The project is split into a Java/Spring Boot backend, a React/Vite frontend, and database scripts.

```

CodeExam/
├── backend/                                Spring Boot API
│   ├── pom.xml                            Maven dependencies and build settings
│   └── src/
│       ├── main/java/com/example/exam/
│       │   ├── OnlineExamApplication.java Application entry point
│       │   ├── api/                        REST API controllers
│       │   ├── service/                   Business logic
│       │   ├── repository/               Database access interfaces
│       │   ├── model/                    JPA database entities
│       │   ├── dto/                      Request/response data objects
│       │   ├── mapper/                   Entity-to-DTO conversion
│       │   ├── config/                   Security, JWT, and app configuration
│       │   ├── execution/                Code execution providers
│       │   ├── ai/                       AI question-generation support
│       │   ├── controller/              Code-execution endpoints
│       │   └── util/                     Shared parsing and security helpers
│       ├── main/resources/
│       │   └── application.properties Server, DB, JWT, AI settings
│       └── test/                          Backend tests
├── frontend/
│   ├── package.json                       Workspace scripts
│   └── frontend/                          React/Vite web application
│       ├── src/
│       │   ├── App.tsx                   Client-side routes
│       │   ├── pages/                    Admin, student, auth, home pages
│       │   ├── components/               Reusable UI and question components
│       │   ├── services/                 HTTP API calls
│       │   ├── context/                  Authentication state
│       │   ├── lib/                      Axios client and interceptors
│       │   ├── hooks/, types/, utils/    Shared frontend code
│       │   └── main.tsx                   React bootstrap file
│       ├── public/                       Static browser assets
│       ├── vite.config.ts                 Vite server and API proxy settings
│       └── package.json                   Frontend dependencies/scripts
├── database/
│   ├── schema_mysql.sql                  MySQL schema reference
│   ├── schema.sql                       Alternative schema reference
│   └── clean_data.sql                    Data cleanup script

```

		└─ README.md	Database setup notes
		└─ start-system.bat	Starts backend and frontend locally
		└─ README.md	Project setup and usage guide
		└─ project-documentation.html	General project documentation source

What each backend layer does

Folder	Responsibility
api	Receives HTTP requests such as login, create exam, add question, results, and student actions.
service	Applies business rules, coordinates database operations, evaluation, authentication, and leaderboards.
repository	Uses Spring Data JPA to read and write database entities.
model	Defines Java classes mapped to database tables.
dto and mapper	Defines safe API payloads and converts database models into those payloads.
config	Configures JWT authentication, access control, CORS, and initial admin setup.

What each frontend area does

Folder	Responsibility
pages	Complete screens for admin management, student exams, login, results, and profiles.
components	Reusable visual blocks such as the navigation bar, tables, forms, editor, and modals.
services	Frontend functions that call backend APIs.
context	Stores current user and authentication state across the app.
lib	Configures Axios, API base URL, JWT request headers, and login redirect behavior.

Generated for the Smart Coding Exam System.